MIZAN - TEPI UNIVERSITY

TEPI CAMPUS

SCHOOL OF COMPUTING AND INFORMATICS

DEPARTMENT OF INFORMATION SYSTEMS

GROUP ASSIGNMENT OF COMPUTER ORGANAZITION AND ARCHITECTURE

COURSE CODE : INSY2054

GROUP 3 Section A

NO	NAME	ID
1	GEBRIE ADANE	MTUUR/7832/17
2	EYASU WOLDE	MTUUR/8432/17
3	BINYAM TEWODROSE	MTUUR/9411/17
4	GETAHUN KEBEDE	MTUUR/8935/17
5	BINYAM MATHEWOS	MTUUR/8877/17
6	GETANEH MATHEWOS	MTUUR/8796/17
7	EALSHADEY YIRGALEM	MTUUR/8384/17
8	HIWOT TESFAYE	MTUUR/7978/17
9	FIREHIWOT ENDALE	MTUUR/8700/17
10	HERMELA NGUSSIE	MTUUR/8208/17

Submitted To : Mr. Haile

Submission Date: 19/8/2018 E.C

Contents

Introduction.....i

Chapter 6 Functional organization 1

6.1 Implementation of simple data paths.....3

6.2 Control unit: hardwired realization vs. micro programmed realization 7

6.3 Instruction Pipelining 9

6.4 Introduction to instruction level Parallelism Processing.....12

Conclusion 16

References 17

Introduction

Parallel processing and pipelining are fundamental techniques used to enhance the performance of modern computer systems. Traditionally, performance improvements have been achieved by employing multiple processors that operate concurrently, as seen in symmetric multiprocessors (SMP). Parallel processing involves using multiple processing elements—such as multiple ALUs, processors, or networked computers—simultaneously to solve a problem, thereby increasing throughput. Computer architectures are generally classified into four categories: SISD (traditional uniprocessor), SIMD (suitable for data-parallel tasks), MISD (used in fault-tolerant systems), and MIMD (common in modern multicore and distributed systemsas completed., data dependency, and branch difficulties. Data dependency can be managed through hardware interlocking, operand forwarding, or delayed load techniques. Branch difficulties can be mitigated by prefetching, branch target buffers, or branch prediction.The data path—comprising components such as the ALU and registersprocesses input data to generate outputs. The control unit, which directs these operations, can be implemented either as hardwired circuitry (common in RISC architectures) or as a micro-programmed control unit using a control memory (common in CISC architectures).

Chapter 6

Functional organization

- ***Functional organization*** is a type of organizational structure that uses the principle of specialization based on function or role.

- It allows decisions to be decentralized since issues are delegated to specialized persons or units, leaving them the responsibility of implementing, evaluating, or controlling the given procedures or goals. In computer engineering, computer architecture is a set of rules and methods that describe the functionality, organization, and implementation of computer systems. In other definitions computer architecture involves instruction set architecture design, micro architecture design, logic design, and implementation.The purpose is to design a computer that maximizes performance while keeping power consumption in check, costs low relative to the amount of expected performance, and is also very reliable. The implementation involves integrated circuit design, packaging, power, and cooling. Optimization of the design requires familiarity with compilers, operating systems to logic design, and packaging. An instruction set architecture (ISA) is the interface between the computer's software and hardware and also can be viewed as the programmer's view of the machine.

Computers do not understand high-level programming language such as Java, C++, or most programming languages used. A processor only understands instructions encoded in some numerical fashion, usually as binary numbers.

▪ Software tools, such as compilers, translate those high-level languages into instructions that the processor can understand.

▪ Besides instructions, the ISA defines items in the computer that are available to a program. Examples data types, registers, addressing modes, and memory. Instructions locate these available items with register indexes (or names) and memory addressing modes. 5 Implementation is usually not considered architectural design, but rather hardware design engineering.

▪ Implementation can be further broken down into several steps:

✓**Logic Implementation** designs the circuits required at a logic gate level

✓**Circuit Implementation** does transistor-level designs of basic elements (gates, multiplexers, latches etc.) as well as of some larger blocks (ALUs, caches etc.) that may be implemented at the log gate level, or even at the physical level if the design calls for it.

6.1 Implementation of simple data paths

A Data path is a critical component within a processor that handles data processing operations. It works in conjunction with a control unit to form the CPU (central processing unit) of a computer system.

- **Data path** –is the collection of state elements, computation elements, and interconnections that together provide a conduit for the flow and transformation of data in the processor during execution.

- **Simple data path** Simple data path components include memory (stores the current instruction), PC or program counter (stores the address of current instruction), and ALU (executes current instruction). The data path is the "brawn" of a processor, since it implements the fetch-decode-execute cycle. The general discipline for data path design is considered:
  - To determine the instruction classes and formats in the ISA, To design data path components and interconnections for each instruction class or format, and
  - To compose the data path segments designed in Step 2 to yield a composite data path.

Components of a Simple Data path:

A. ALU (Arithmetic Logic Unit) : The ALU executes the current instruction and perform arithmetic and logic operation. ALU modern CPU contains a control unit and a set of registers. Most of the operations are performed by one or more ALU's, which load data from the input register.

B. Register : are like gates here through which signals are further sent to components to do the micro-operation. Here are five registers used to store in-out signal data:

- **Program Counter** The program counter (PC) is a CPU register in the computer processor which has the address of the next instruction to be executed from memory.
- ***Instruction Register*** In computing, an instruction register (IR) is the part of a CPU's register that holds the instruction currently being executed or decoded.
- **Memory Address Register** The Memory Address Register (MAR) is the CPU register that either store the memory address from which data will fetch from the CPU or the address to which data will be sent and stored.
- **Memory data register** The memory data register (MDR) is the register in a computer's processor, or

central processing unit (CPU) that stores the data being transferred to and from the immediate access storage.

- **General purpose register:** General-purpose registers are used to store temporary data within the microprocessor, and it is a multipurpose register.

C. **Bus** is parallel electrical wires with multiple hardware connections. Therefore a bus is a communication system that transfers data between components inside a computer or between computers. When we would have different kinds of buses, different types of bus organizations will take place, such as**:**

- ✓ **One Bus organization** In one bus organization, a single bus is used for multiple purposes. A set of general-purpose registers, program counters, instruction registers, memory address registers (MAR), and memory data registers (MDR) are connected with the single bus.
- ✓ **Two Bus organization** In two bus organizations, there are two buses, and the general-purpose register can read/write from both the buses. In this case, two operands can be fetched at the same time because of the two buses**.**
- ✓ **Three Bus organization** In three bus organizations, we have three buses, OUT bus1, OUT bus2, and an IN bus. From the out buses, we can get the operand that can come from the general-purpose register and evaluated

in ALU, and the output is dropped on In Bus to be sent to respective registers.

D. Memory : is used to store instructions and data that the processor needs to access. It can be divided into different types, such as main memory(RAM) and secondary storage (Hard disk, SSD).

E. Control Unit: is responsible for managing the overall operation of the data path. Control unit generates all the signals (in-out) based on which all other component works

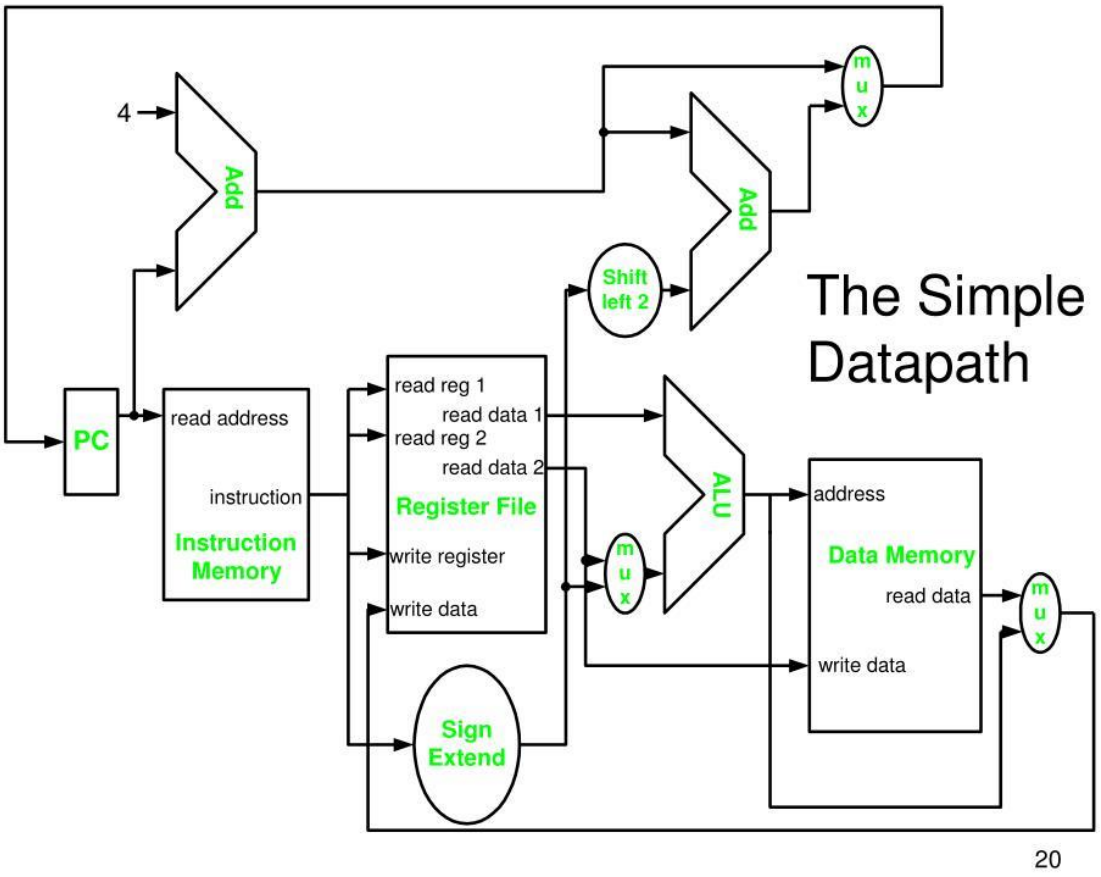


Figure 1

6.2 Control unit: hardwired realization vs. micro programmed realization

The Hardwired and Micro programmed control unit generates the control signals to fetch and execute instructions. The fundamental difference between hardwired and micro programmed control unit is that hardwired is a circuitry approach whereas, the micro program control unit is implemented by programming. The hardwired control unit is designed for the RISC(reduced instruction set computer) On the other hand, the micro programmed control unit was designed for the CISC(complex instruction set computer) Both Hardwired and Micro programmed control unit are designed to 'generate' the control signals. The control signals operate the functioning of the processor's hardware. It decides what operation has to be performed, what must be the sequence of the operations performed by the processor, in what time an operation must be executed and so on. the hardwired control unit generates the control signals to execute the instructions in a proper sequence and at the correct time.The hardwired control unit is created with the hardware; it is a circuitry approach. It is designed for the RISC style instruction set. Micro programmed Control Unit?

Micro programmed control unit also produces the control signal but using the programs.

The program that creates the 'control signals' is called Micro program. This micro program (tells the control unit how to execute each instruction) is placed on the processor chip which is a fast memory, it is also called control memory or control store.

Each microinstruction is 'n' bit word. Each control signal differs from other control signal depending on the bit pattern of the control word

Microprogrammed Control vs Hardwired Control

Microprogrammed Control	Hardwired Control
It is a microprogram in control store that generates control signals.	It is the sequential circuit that generates control signals.
Speed of operation is low, because it involves memory access.	Speed of operation is high.
Changes in control behavior can be implemented easily by modifying the microinstruction in the control store.	Changes in control unit behavior can be implemented only by redesigning the entire unit.

Table 1

6.3 Instruction Pipelining

Pipelining is an implementation technique where multiple instructions are overlapped in execution. The computer pipeline is divided into stages. Each stage completes a part of an instruction in parallel. The stages are connected one to the next to form a pipe - instructions enter at one end, progress through the stages, and exit at the other end. Pipelining refers to the technique in which a given task is divided into a number of subtasks that need to be performed in sequence. Instruction Pipeline An instruction pipeline reads consecutive instructions from memory while previous instructions are being executed in other segments. Consider a computer with an instruction fetch unit and an instruction execution unit forming a two segment pipeline. A FIFO (First In First Out) buffer can be used for the fetch segment. Thus, an instruction stream can be placed in a queue, waiting for decoding and processing by the execution segment. This reduces the average access time of memory for reading instructions. Whenever there is space in the buffer, the control unit initiates the next instruction fetch phase. There are three major difficulties that causes instruction pipeline to deviate from its normal operation and those are:

1. Resource conflicts caused by access to memory by two segments at the same time.

2. Data dependency conflicts arise when an instruction depends on the result of a previous instruction, but this result is not yet available

3. Branch difficulties arise from program control instructions that may change the value of program counter

❖ **Methods to handle data dependency:**

• Hardware interlocks are circuits that detect instructions whose source operands are destinations of prior instructions. Detection causes the hardware to insert the required delays without altering the program sequence.

• Operand forwarding uses special hardware to detect a conflict and then avoid it by routing the data directly from the ALU output to the next instruction through special paths between pipeline segments. Delayed load is a procedure that gives the responsibility for solving data conflicts to the compiler. The compiler is designed to detect a data conflict and delay the loading of the conflicting data by inserting no-operation instructions If the compiler cannot find any independent instruction.

❖ **Methods to handle branch instructions**: Prefetching the target instruction in addition to the next instruction allows either instruction to be available.

A branch target buffer is an associative memory included in the fetch segment of the branch instruction that stores the target instruction for a previously executed branch.

Branch prediction uses some additional logic to guess the outcome of a conditional branch instruction before it is execute

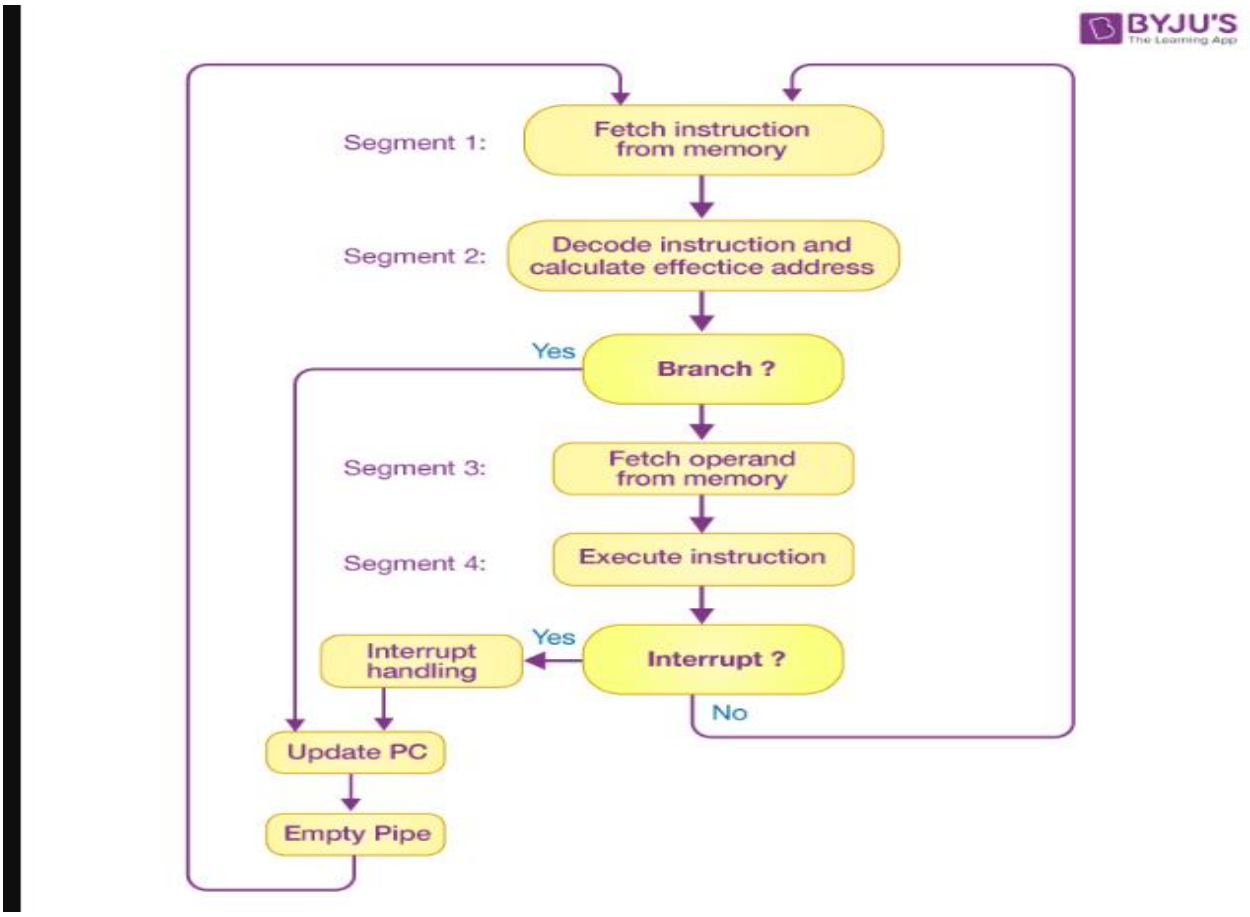


Figure 2

6.4 Introduction to instruction level Parallelism Processing

A traditional way to increase computer system performance is to use multiple processors that can execute in parallel to support a given workload. The most common multiple-processor organizations is symmetric multiprocessors (SMPs) SMP consists of multiple similar processors within the same computer, interconnected by a bus or some sort of switching arrangement.

Parallel processing/computing, uses multiple processing elements simultaneously to solve a problem. The processing elements can be diverse and include resources such as a single computer with multiple processors, several networked computers, specialized hardware, or any combination of the above. The following are important points about parallel processing: A parallel processing system is able to perform concurrent data processing to achieve faster execution time The system may have two or more ALUs and be able to execute two or more instructions at the same time Also, the system may have two or more processors operating concurrently The goal is to increase the throughput – the amount of processing that can be accomplished during a given interval of time Computers can be classified as:

Single instruction stream, single data stream – SISD

Single instruction stream, multiple data stream – SIMD
Multiple instruction stream, single data stream – MISD
Multiple instruction stream, multiple data stream – MIMD

A.Single instruction stream, single data stream – SISD

Traditional uniprocessor architecture where one CPU executes one instruction at a time.

Single control unit issues one instruction stream sequentially. Only one data stream is processed, so operations occur on a single set of data values. No parallelism; computation is strictly serial, making it simple but slower for large tasks.

Common in classic Von Neumann systems and basic single-core processors.

B. Single instruction stream, multiple data stream – SIMD

One instruction is issued by the control unit but executed on multiple data elements simultaneously. Uses multiple processing units, each operating on a different piece of data in parallel.

Ideal for data-parallel tasks like graphics, image processing, matrix operations, and scientific simulations. Offers high

performance and efficiency for large, structured datasets due to simultaneous operations.

C. Multiple instruction stream, single data stream – MISD

Multiple processors execute different instructions but operate on the same data stream. Used where fault tolerance, redundancy, or reliability is critical (each processor checks the other).

Not common in general-purpose computing due to limited practical applications. Suitable for real-time, safety-critical systems such as military, aerospace, and mission-critical control systems. Often implemented as pipelined or redundant computing architectures rather than general processors.

D.Multiple instruction stream, multiple data stream – MIMD

Multiple independent processors, each executing its own instruction stream on different data sets simultaneously. Supports both task parallelism and data parallelism, making it highly flexible for many workloads.

Common in modern multicore CPUs, clusters, distributed systems, and cloud computing architectures.

Offers high scalability and performance, ideal for scientific computing, large databases, AI, and multitasking environments.

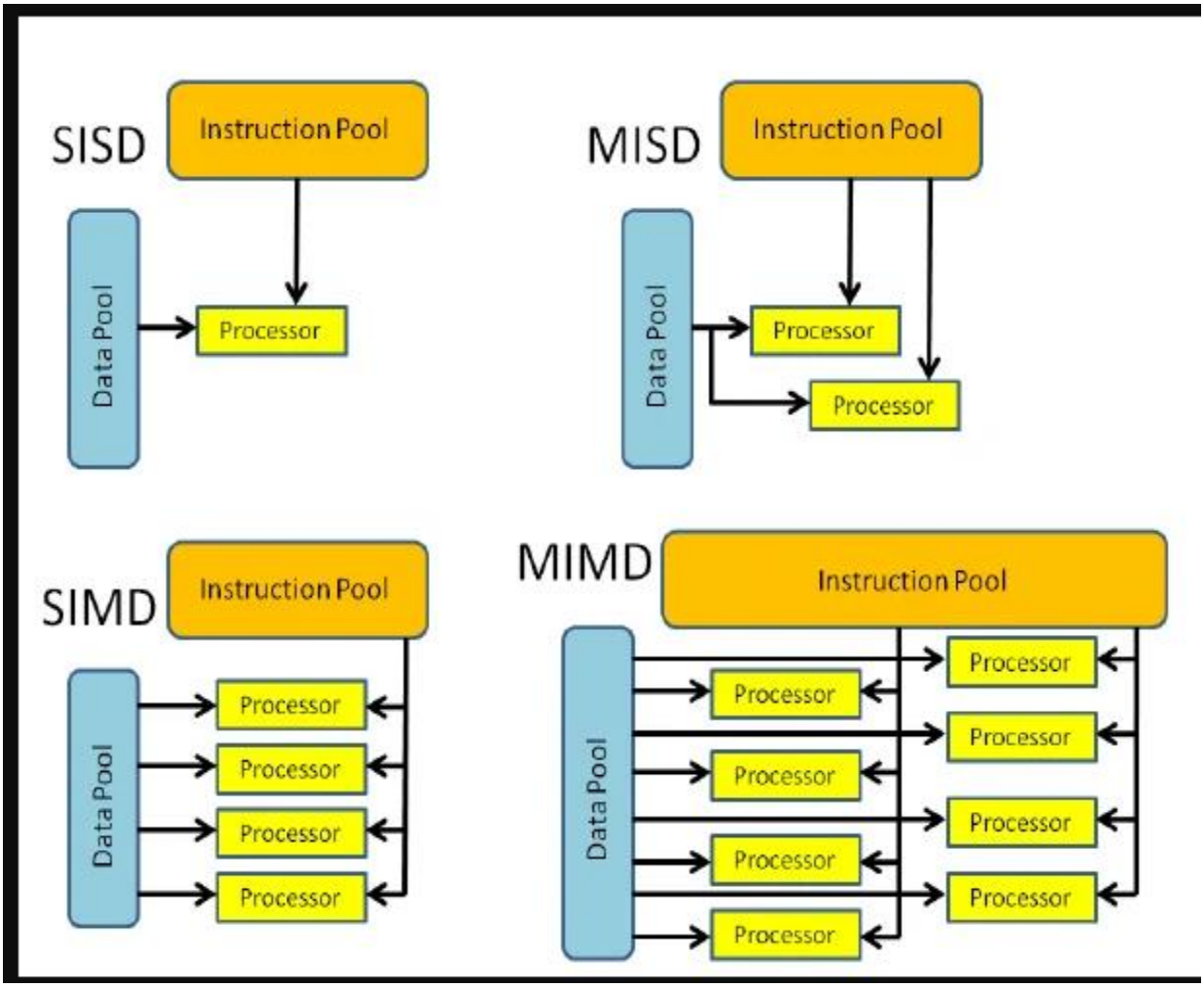


Figure 3

Conclusion

In summary, parallel processing and pipelining are essential strategies for achieving high performance in contemporary computing. Parallel processing, through MIMD and SIMD architectures, enables simultaneous execution of multiple instruction and data streams, significantly improving throughput for scientific, AI, and multitasking workloads.

Pipelining, despite challenges such as resource conflicts, data dependencies, and branch difficulties, increases instruction throughput by overlapping execution stages.

Techniques like operand forwarding, delayed load, and branch prediction effectively mitigate these pipeline hazards.

Furthermore, the design of the data path and control unit plays a critical role in overall processor efficiency.

Hardwired control units offer speed and simplicity for RISC processors, while micro-programmed control units provide flexibility for complex instruction sets.

Understanding these trade-offs is crucial for designing balanced, high-performance computer systems.

References

1. Chapter 6: Functional Organization – Introduction to Parallel Processing. (Discusses SISD, SIMD, MISD, MIMD classifications and symmetric multiprocessors).

2. Instruction Pipeline and Pipelining Techniques. (Covers pipeline stages, FIFO buffer, resource conflicts, data dependency, branch difficulties, operand forwarding, delayed load, prefetching, branch target buffer, and branch prediction).

3. Data Path and Control Unit Organization. (Describes data path logic, ALU, registers, hardwired vs. micro-programmed control unit, RISC vs. CISC, and control memory/microprogram).